



INDEPENDENT SMART CONTRACT SECURITY REVIEW

Ascnt v1 Core: One-Day Complimentary Security Review

A best-effort assessment of Ascnt v1 Core performed as a one-day complimentary security review, prioritizing high-signal risks in dynamic-fee calculation, accumulator accounting, and split-route behavior.

PREPARED FOR

Ascnt Labs

STATUS

Initial Review

VERSION

v1.0

DATE

2026-08-27

VULSIGHT

PREPARED BY

Vulsight

WEB

vulsight.com

CONTACT

info@vulsight.com

Document Control

CLIENT**Ascnt Labs****PROJECT****Ascnt v1 Core**

Ethereum mainnet target; reviewed package is pre-deployment and contains no deployment script or configured live pool

Contents

- 01** Executive Summary
- 02** Scope and Methodology
- 03** Severity Classification
- 04** Findings Overview
- 05** Detailed Findings
- 06** Appendix and Disclaimer



VULSIGHT

Executive Summary

Overall Risk: High for production readiness; no direct principal-loss path was identified within the reviewed scope

This report reflects a complimentary one-day review of selected Ascnc v1 Core contracts and documentation. The work focused on fee correctness, accumulator behavior, routing invariants, and defects that could be substantiated with clear source evidence and meaningful impact.

The review identified one High, one Medium, and two Low findings. No direct principal-loss path was identified within the reviewed scope, but several stated fee-model guarantees do not hold under adversarial routing and zero-liquidity edge cases. Given the time-boxed review depth, these findings represent high-confidence observations rather than exhaustive coverage. The High-severity issue should be treated as a pre-launch blocker for the protocol's fee-model claims.

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	1	1	2	0

ITEM	VALUE
Source	ascnclabs/v1-core-audit
Commit	87ddf6ba464f00548c449681755050343df89322
Language / Stack	Solidity 0.8.26 / Uniswap v4 hooks
Network	Ethereum mainnet target; reviewed package is pre-deployment and contains no deployment script or configured live pool
Review Approach	- Complimentary one-day review of selected Ascnc v1 Core source and documentation. - Best-effort validation against source references, tests, stated invariants, and documented accepted risks. - Severity calibrated to the time-boxed review depth, exploitability, and documented trust boundaries.

Limitations

This complimentary one-day initial review was best-effort and focused on selected smart-contract paths and repository documentation; it was not a comprehensive line-by-line audit. It excluded live deployment validation, on-chain configuration, off-chain governance operations, and execution of the Foundry test suite. Third-party dependencies under `lib/` were reviewed only at their integration boundaries. Effects that depend on exact rounding should be reconfirmed with executed tests against the pinned dependency set before report closure.

Scope and Methodology

IN SCOPE

- **src/SimHook.sol**: Primary fee model, accumulator lifecycle, min-fee ramp, and JIT lock logic.
- **src/AscntBaseHook.sol**: Protocol-fee split and take flow, authority gates, and pause behavior.
- **src/AscntGovernance.sol**: Global authority model, subscriber updates, protocol-fee configuration, and lifecycle controls.
- **src/HookFactory.sol**: Hook deployment and provenance registration path.
- **src/lib/SwapSimulator.sol**: Fee-free simulation engine used by SimHook pricing.
- **src/lib/HookMath.sol**: Impact calculation, accumulator decay, and effective min-fee math.
- **src/lib/SafeCast.sol**: Saturating casts used by fee and accumulator accounting.

OUT OF SCOPE

- Third-party dependencies under `lib/`, including Uniswap v4 core/periphery, OpenZeppelin, and forge-std.
- Deployment scripts, timelock operations, multisig procedures, monitoring, and other operational controls.
- Live pool configuration, production addresses, and on-chain economic usage.
- Frontend, SDK, routing, quoting, and off-chain execution infrastructure.
- Formal verification and full economic modeling across real market states.

Methodology

- Performed a time-boxed best-effort review of selected source files in `src/` against the repository's invariants, known-issue register, and test intent.
- Traced the swap lifecycle from `\_beforeSwap` simulation and fee quotation through `\_afterSwap` settlement and accumulator updates.
- Validated candidate findings for concrete source grounding, exploitability, economic effect, and overlap with documented accepted risks.
- Prioritized high-confidence issues that materially weaken fee-model or accounting guarantees, while excluding speculative hardening observations from the report.

Severity Classification

Severities are assigned from realistic impact and likelihood under the reviewed deployment and documented trust model. Latent risk is called out separately where a roadmap feature or configuration change would materially change impact.

SEVERITY	CLASSIFICATION STANDARD	USE IN THIS REVIEW
CRITICAL	Direct loss of funds, insolvency, or a system-wide failure that can be exploited without special governance or deployment assumptions.	No Critical findings were identified in the current source review.
HIGH	A core security or pricing guarantee fails under realistic adversarial use, with material effect on protocol behavior or fee capture.	Used where the implementation undermines the hook's primary fee-model objective in live trading conditions.
MEDIUM	Meaningful security or accounting degradation that requires specific state setup or market structure, but remains live and exploitable.	Used for accumulator defects that are reachable in production logic but depend on topology, liquidity layout, or execution shape.
LOW	Limited but real correctness, accounting, or hardening issue with bounded impact or narrower reachability.	Used for issues that weaken pricing accuracy, metering fidelity, or documented invariants without directly enabling large-scale loss on their own.
INFORMATIONAL	Improves clarity, safety posture, monitoring, future readiness, or specification quality.	No standalone Informational findings are included in this draft; documentation gaps are folded into the relevant technical findings.

VULSIGHT

Findings Overview

ID	TITLE	SEVERITY	STATUS	COMPONENTS
ASCNT-H-01	Fee is derived in impact space but charged on token notional	HIGH	Open	src/SimHook.sol, src/lib/SwapSimulator.sol
ASCNT-M-01	Exact-zero settlement guard breaks accumulator path-to-state consistency	MEDIUM	Open	src/SimHook.sol
ASCNT-L-01	Exact-input fee-free quoting and fee-bearing execution diverge from published split-neutrality claims	LOW	Open	src/SimHook.sol, src/lib/SwapSimulator.sol
ASCNT-L-02	Sub-pip moves are discarded with no fractional carry	LOW	Open	src/lib/HookMath.sol, src/SimHook.sol

Status Summary

TOTAL FINDINGS	4	OPEN	4	ACKNOWLEDGED	0	RESOLVED	0
----------------	---	------	---	--------------	---	----------	---

VULSIGHT

Detailed Findings

ASCNT-H-01 Fee is derived in impact space but charged on token notional

src/SimHook.sol, src/lib/SwapSimulator.sol

HIGH

STATUS Open

LIKELIHOOD

Medium; concentrated-liquidity profiles are common market structure, and adversarial routers can intentionally split flow.

IMPACT

LPs and the protocol can collect materially less fee than the design intends, especially when flow is split across variable-liquidity books.

DESCRIPTION

`SimHook._beforeSwap` prices a swap from simulated pre/post `sqrtPriceX96` endpoints and discards the simulator's token deltas. Uniswap v4 then applies the quoted LP fee to token notional at each step of the real swap.

Those two domains coincide only under uniform liquidity. In a concentrated-liquidity book, the same aggregate price movement can traverse materially different notional profiles depending on how flow is split. This means a trader can reduce effective fees by partitioning flow across legs or route segments that achieve a similar end state while paying less fee than the one-shot path the protocol's invariant language suggests.

The defect is structural: the simulator already computes the information needed to align pricing with notional, but the hook currently relies only on endpoint impact. As a result, the implementation weakens the protocol's primary design goal of charging toxic flow in proportion to its economic footprint.

PROOF OF CONCEPT

- Construct a pool with materially uneven liquidity density across the price path.
- Compare one large exact-input swap against a split route that reaches a similar net inventory state through smaller legs.
- Observe that the charged fee is derived from endpoint impact, while v4 collects it on the leg notionals actually traversed.
- The split route can realize a lower aggregate fee than the protocol's one-shot fee model implies, despite causing comparable aggregate inventory movement.

RECOMMENDATION

Rework pricing so the charged fee is not derived solely from endpoint impact. The cleanest direction is to use the simulator's returned token deltas, or a leg-weighted equivalent, so the quoted fee is aligned with the notional that v4 will actually charge. If this is intentionally deferred, weaken the path-additivity and no-split-advantage claims in public documentation and monitoring guidance.

REFERENCES

- src/SimHook.sol:293-317
- src/lib/SwapSimulator.sol:16-23
- src/lib/SwapSimulator.sol:57-58
- src/lib/SwapSimulator.sol:191-203
- docs/invariants.md:30-37

[BACK TO FINDINGS OVERVIEW](#)

ASCNT-M-01 Exact-zero settlement guard breaks accumulator path-to-state consistency

src/SimHook.sol

MEDIUM

STATUS Open

LIKELIHOOD

Low to Medium; requires zero-liquidity topology and non-standard execution sequencing, but the logic is live in production code.

IMPACT

An attacker can inject or erase standing cumulative impact without paying for a matching economic move, distorting later fees for other users.

DESCRIPTION

`SimHook._afterSwap` treats any swap that settles with `amount0 == 0` and `amount1 == 0` as zero impact, even if spot moved materially across an empty region before returning to a settled zero-delta state. This shortcut is intended to avoid booking a purely unpaid traversal, but it creates a deeper accounting inconsistency.

The accumulator is path-based while spot price is a state variable. Once exact-zero traversals are ignored asymmetrically, a closed price loop can leave a nonzero accumulator, and a reverse loop can scrub a standing surcharge without paying for a corresponding unwind. This means later users can face fees that no longer reflect net economic pressure in the book.

The issue is not merely a restatement of the accepted KI-6 saturation scenario. The code path here allows the accumulator to diverge from the end state of the book itself, which is a stricter failure than simply over-booking a real paid move.

PROOF OF CONCEPT

- Build a pool whose current spot sits in a zero-liquidity gap between funded islands.
- Use a dust trade that settles nonzero to book a full traversal into the accumulator.
- Use the reverse exact-zero path to restore spot across the empty region while producing zero settlement deltas.
- The accumulator remains elevated, or can be scrubbed toward zero in the mirror case, even though the net state of the pool no longer supports that meter reading.

RECOMMENDATION

Replace the exact-zero shortcut with logic that preserves path/state consistency. A robust fix should update the accumulator from the realized price path rather than treating zero settlement as automatic zero impact. At minimum, add regression tests for both variants of the defect: impact injection on a net-zero round trip and surcharge scrubbing on the reverse path.

REFERENCES

- src/SimHook.sol:357-375
- docs/invariants.md:62-66
- docs/known-issues.md:131-149
- test/differential/Phase3_SimEngineStructure.t.sol:208-236

BACK TO FINDINGS OVERVIEW

ASCNT-L-01

Exact-input fee-free quoting and fee-bearing execution diverge from published split-neutrality claims

src/SimHook.sol, src/lib/SwapSimulator.sol

LOW

STATUS Open

LIKELIHOOD

Medium; most visible at higher route impact, and present whenever exact-input execution materially diverges from the fee-free replay.

IMPACT

Even under uniform liquidity, exact-input split routes can beat the one-shot quote relative to the published invariant, and corrective exact-input quotes are not uniformly conservative once translated from impact into fee outcomes.

DESCRIPTION

`SwapSimulator` prices exact-input swaps from a fee-free replay, while `_afterSwap` advances state using the smaller realized fee-bearing move. Nothing reconciles those two quantities, and same-block decay is the identity, so the difference persists across sequential legs.

This mechanism is distinct from ASCNT-H-01. It does not depend on variable liquidity or on charging fees against the wrong economic base; it can arise even when liquidity is uniform and impact and notional remain aligned. The divergence comes from using a fee-free simulated endpoint to quote the current leg, then updating state from the smaller movement the real fee-bearing leg actually reaches.

The practical failure is narrower than a broad revenue-loss claim, but still material for client-facing guarantees. Exact-input atomic splits need not remain quote-neutral relative to the one-shot path described in the documentation, and the fee-free impact estimate is not uniformly conservative once translated into fee outcomes on the non-crossing corrective branch. The current invariant and integrator language therefore overstate what the implementation guarantees.

PROOF OF CONCEPT

- Use a constant-liquidity or otherwise uniform-liquidity pool to isolate this issue from ASCNT-H-01.
- Start from a near-zero meter in an unclamped pool and compare one exact-input route against the same gross input split into same-block tranches.
- Each tranche is quoted from a fee-free simulated endpoint while state is later advanced by a smaller fee-bearing realized move.
- The blended split quote can therefore beat the one-shot quote relative to the published invariant, even when liquidity weighting is held constant.
- Separately, establish a nonzero standing imbalance and submit a corrective exact-input swap that remains on the non-crossing healing branch.
- The simulator overstates impact, yet the quoted corrective fee can move downward because the branch uses `2C - p` once translated into fee space.

RECOMMENDATION

Update the documentation and test suite to distinguish impact conservatism from fee conservatism, and narrow invariant #3 so it describes the regime the implementation actually preserves.

Add an end-to-end regression that holds liquidity constant, compares one exact-input route against same-block tranches, and separately exercises the non-crossing corrective branch. That isolates this issue from ASCNT-H-01 and reduces overlap in future triage.

If stronger fee accuracy is required, any remediation should explicitly handle the fee-free quote versus fee-bearing execution mismatch rather than assuming a second-pass quote will converge automatically.

REFERENCES

- src/lib/SwapSimulator.sol:18-23
- src/SimHook.sol:287-322
- src/SimHook.sol:353-375
- src/SimHook.sol:463-483
- docs/invariants.md:30-37
- docs/invariants.md:95-101
- docs/known-issues.md:85-92
- docs/known-issues.md:245-287

BACK TO FINDINGS OVERVIEW

ASCNT-L-02 Sub-pip moves are discarded with no fractional carry

src/lib/HookMath.sol, src/SimHook.sol

LOW

STATUS Open

LIKELIHOOD

Low to Medium; requires repeated very small price moves, but no privileged action.

IMPACT

The accumulator can miss aggregate directional movement when each leg individually rounds below one pip.

DESCRIPTION

`HookMath.calculatePriceImpactCapped` floors price impact to whole pips, and `SimHook._afterSwap` writes that integer result directly into `cumPriceImpact`. The implementation does not retain any fractional remainder between swaps.

This means repeated sub-pip moves can shift the pool while booking zero each leg. The state-based meter therefore cannot defend against an imbalance it never recorded. This is distinct from the low-price degradation already documented in KI-14: here, the same loss of precision can occur at ordinary price levels when the book is deep enough and each leg remains individually below the one-pip threshold.

The effect is bounded, but it weakens the claim that the accumulator fully closes the dust-splitting hole in all normal-price regimes.

PROOF OF CONCEPT

- Operate in a deep pool at ordinary price levels, well above `MIN_USABLE_SQRT_PRICE`.
- Execute a sequence of same-direction trades whose realized price movement is below one pip each.
- Observe that `AfterSwap.priceImpact` rounds to zero on each leg and `cumPriceImpact` does not capture the aggregate directional drift until a leg crosses the threshold.

RECOMMENDATION

Introduce a higher-precision internal accumulator or a fractional carry field so sub-pip movement is not lost between swaps. If that change is intentionally deferred, narrow the documentation around dust-resistance so it does not imply perfect metering below one pip.

REFERENCES

- src/lib/HookMath.sol:13-59
- src/SimHook.sol:366-374
- docs/known-issues.md:370-390

[BACK TO FINDINGS OVERVIEW](#)

VULSIGHT

Appendix

Accepted-Risk Register Reviewed

Behaviors already classified by the repository as accepted risks were not re-reported unless the implementation materially exceeded the documented rationale. KI-1, KI-6, KI-7, KI-8, KI-11, KI-13, and KI-14 were considered during triage; only implementation gaps extending beyond those positions were retained as findings.

This approach preserves actionable signal without inflating documented design tradeoffs into additional vulnerabilities.

Review Depth And Follow-Up

This was a complimentary one-day initial review. The limited window was used to prioritize issues with clear source grounding and meaningful security or economic consequences. The findings are best-effort observations from the reviewed paths and documents, not an exhaustive assessment of every contract, dependency, configuration, or operational process.

The repository's written invariant set is stronger than what the current implementation and tests fully support in two material respects. First, invariant #3 and the KI-1 integrator note currently describe atomic splitting as roughly fee-neutral under the midpoint model, which is too strong once fee-free quoting and fee-bearing execution are composed. Second, the simulator's exact-input bias is accurately conservative in impact space, but that language does not carry over unchanged to fee outcomes on the corrective branch.

A further nuance is worth capturing in the same documentation pass: `calculatePriceImpactCapped` is an endpoint reading, not an additive route coordinate. Formula-level midpoint tests that assume leg lengths sum exactly therefore do not, by themselves, prove split neutrality across composed price paths. That observation is useful for narrowing the wording of invariant #3 and for designing future end-to-end coverage, even though it did not justify adding a separate client finding in this draft.

These items are better addressed through documentation and regression-test updates than as standalone findings. They should be corrected before the protocol relies on those statements in deployment or integration guidance.

Settlement Boundary Follow-Up

One additional settlement-boundary hardening point is worth preserving as follow-up, but it is not strong enough for the main findings set. `_takeProtocolFeeOnAfterSwap` proves that the returned treasury take fits on its own, but that argument does not by itself prove that v4's later recomposition step, `swapDelta - hookDelta`, still has signed 128-bit headroom at the extreme boundary.

Within the reviewed production envelope this appears unreachable at Ethereum-scale token supplies, so it is better treated as a test-coverage and hardening follow-up than as a client finding. If the settlement path is revised, add a regression that exercises the full recomposition step near the signed 128-bit boundary rather than testing the hook helper in isolation.

Production Readiness Note

The repository itself opens with a warning that the contracts are unaudited and not ready for production use. The findings in this review reinforce that warning: several issues do not require privileged governance failure, and at least one High-severity defect cuts directly against the protocol's core fee-model promise. Remediation and retesting should precede any production deployment.

Disclaimer

This report reflects a complimentary, one-day, time-bound initial review of selected Ascnet v1 Core smart-contract code paths and repository documentation. It is not a comprehensive audit, not a guarantee that the system is free of defects, and should not be interpreted as financial, legal, or operational advice.

Within the limited review window, work prioritized high-confidence issues affecting fee correctness, accumulator accounting, and routing behavior. The absence of additional findings does not imply exhaustive coverage. Third-party dependencies, live deployment validation, off-chain operations, external execution infrastructure, economic modeling, and formal verification were excluded and should be assessed separately before production launch.